

Coordinating Distributed Energy Resources Behind One Meter

Theory notes for `home-energy-optimizer`

Abstract

A home with solar panels, batteries, a water heater, a heat pump and an electric vehicle has several distributed energy resources (DERs) behind one meter. Planning them together, so that the electricity bill plus the occupants' discomfort is as small as possible, couples them only through the meter, where energy bought costs more than energy sold. These notes state that problem and the method `home-energy-optimizer` uses to solve it. Each DER is planned by its own solver — a linear, quadratic or dynamic programme, or any other method — provided it answers three queries of a fixed form. A *coordinator* then makes the DERs' plans agree at the meter. The default coordinator, Dantzig–Wolfe, blends the DERs' plans in a small linear programme and bounds how far the result can be from the best plan; the alternative, ADMM, needs no linear programme and steers the DERs with one shared price, but gives no such bound. The result is a way to make DERs from different makers interoperate within a house: together they reach a better outcome than each would alone, and the saving is shared so that each is fairly rewarded for its part. We also describe how a plan is executed and the prices it comes with. Every number quoted is reproducible with a script in `bench/`.

How to read these notes. Section 1 states the problem as equations. Section 2 defines the interface between a DER and a coordinator, and the solvers behind it. Sections 3 and 4 give the two coordinators, each as an optimisation problem and a step-by-step algorithm. Section 5 derives the prices a plan comes with, Section 6 how a plan reaches a real home, and Section 7 how its savings are shared. Section 8 compares the coordinators and Section 9 lists what is and is not guaranteed. Table 1 collects the notation. The appendices hold the device models (A), the device solvers (B), the numerical evidence with the script behind every figure and table (C), and one proof (D). Code is cited by module within the package `home_energy_optimizer` (e.g. `dw.coordinator`), and scripts by path (e.g. `bench/run_planners.py`).

Contents

1	The problem	2
2	The device interface	5
3	The first coordinator: Dantzig–Wolfe	7
4	The second coordinator: ADMM	10
5	What a plan comes with: prices	13
6	From plan to action	14
7	Who saves what	15
8	The two coordinators compared	16

<i>Problem</i> (Section 1)		<i>Dantzig-Wolfe</i> (Section 3)	
$t, n, \Delta t$	slot $t = 0, \dots, n - 1$ of Δt hours	$\mathcal{K}, \mathcal{J}_k$	bidding devices; offers of device k
$c_t^{\text{imp}}, c_t^{\text{exp}}$	import, export price [cur/kWh]	p^{kj}, f_{kj}	offer j : plan, private cost
ℓ_t, g_t	household load, PV output [kW]	w_{kj}	weight on offer j
$d_t = \ell_t - g_t$	inflexible net demand [kW]	$\mathcal{B}; c_t^b, e_t^b, s_t^b$	LP batteries; charge, discharge, energy
z_t	meter flow, + for import [kW]	$z_t^\pm, o_t^\pm$	import/export within, beyond a limit
χ_t	PV curtailed [kW]	π_t, σ_k	duals: meter price, device price
$\bar{z}^{\text{imp}}, \bar{z}^{\text{exp}}$	import, export limit [kW]	$L(\pi), L^{\text{best}}$	lower bound on J^* ; best so far
c^{br}	price beyond a limit [cur/kWh]	ϵ, θ	relative tolerance; price smoothing
$m_t(z), M_t(y)$	meter cost; after curtailment [cur]	<i>ADMM</i> (Section 4)	
k, m	device $k = 1, \dots, m$	i, N	terminal $i = 1, \dots, N$
$p_k \in \mathbb{R}^n$	plan of device k , + drawing [kW]	p_i, f_i	terminal plan, cost
$\mathcal{P}_k$	plans device k can run	$\bar{p}_t$	average imbalance [kW]
$f_k(p)$	private cost of plan p [cur]	u	scaled price (price = ρu)
J, J^*	total cost, its minimum [cur]	r_p, r_d	imbalance, movement residuals
<i>Interface</i> (Section 2)		ϵ^{abs}	stopping tolerance [kW]
$\mathcal{Q}_k$	the device query (9)	<i>Prices</i> (Section 5)	
PR, PX, BR	price resp., proximal step, best resp.	λ_t	value of a stored kWh [cur/kWh]
π, π^+, π^-	price; for drawing, supplying	$V_t(s)$	value function of a DP
v, ρ	target plan [kW], its weight	$\text{POL}_t(s)$	policy of a DP
δ	residual load: all but device k [kW]	π_t^{meter}	cost of one more kWh of demand

Table 1: Notation. Device parameters (capacities, efficiencies, temperatures) are defined with the device models in Section 1.2 and Appendix A. $[x]^+ = \max(x, 0)$ and $[x]^- = \max(-x, 0)$; $\|\cdot\|$ is the Euclidean norm over all slots (and terminals); “cur” is a currency unit.

9 What is guaranteed, and what is not	17
10 Related work	18
A Device models	20
B Device solvers	21
C Numerical evidence	22
D Curtailment and the price devices see	26

1 The problem

1.1 Time and data

The horizon is split into n slots of Δt hours, $t = 0, \dots, n - 1$ (typically $\Delta t = 0.25$ h over 24–48 h). A slot at power p moves $p\Delta t$ kWh. The forecasts are the import and export prices c_t^{imp} and c_t^{exp} , the household load ℓ_t and the PV output g_t . Their difference $d_t = \ell_t - g_t$ is the *inflexible net demand*: what the meter would see with no controllable device. A device may need forecasts of its own — hot-water draw for a water heater, outdoor temperature for a heat pump — which only it uses.

1.2 Devices

Device k ($k = 1, \dots, m$) draws power $p_{k,t}$ from the meter in slot t , negative when it supplies; its *plan* is $p_k = (p_{k,0}, \dots, p_{k,n-1}) \in \mathbb{R}^n$. Everything else about it is private, and summarised by two objects:

- $\mathcal{P}_k \subset \mathbb{R}^n$, the plans it can run, given its physics and its state now;
- $f_k : \mathcal{P}_k \rightarrow \mathbb{R}$, its *private cost*: everything except the electricity bill — discomfort, the value of energy left in store at the end of the horizon, a missed charging goal.

Each device modelled here has one state variable, which evolves with its power from a measured initial value. The constraints below hold for every slot t ; Appendix A derives them.

Battery (state: stored energy s_t ; reserve $\underline{S}$ and capacity $\bar{S}$; power limits $\bar{P}_c, \bar{P}_d$; one-way efficiencies η_c, η_d ; terminal price π_T):

$$\begin{aligned} \text{dynamics:} \quad & s_{t+1} = s_t + \Delta t \psi(p_t), \quad \psi(p) = \eta_c p \text{ if } p > 0, \quad \psi(p) = p/\eta_d \text{ if } p \leq 0, \\ \text{constraints:} \quad & -\bar{P}_d \leq p_t \leq \bar{P}_c, \quad \underline{S} \leq s_t \leq \bar{S}, \\ \text{private cost:} \quad & f(p) = \pi_T (s_0 - s_n). \end{aligned} \quad (1)$$

Charging stores only η_c of the energy drawn; each kWh delivered drains $1/\eta_d$ from the store. Energy drawn down over the horizon is charged at π_T (the lowest import price by default), so energy left in store is worth π_T per kWh.

Water heater (state: water temperature T_t ; element power P ; thermal capacitance C ; set-point T^*):

$$\begin{aligned} \text{dynamics:} \quad & T_{t+1} = T_t + \frac{\Delta t}{C} (P\alpha_t - Q_t^{\text{out}}(T_t)), \\ \text{constraints:} \quad & a_t \in \{0, 1\} \text{ (or } [0, 1] \text{ if modulating)}, \quad \alpha_t = \min\{a_t, \bar{\alpha}_t(T_t)\}, \quad p_t = P\alpha_t, \\ \text{private cost:} \quad & f(p) = \Delta t \sum_t \pi_K^{\text{wh}} [T^* - T_{t+1}]^+ + C\bar{c} [T^* - T_n]^+. \end{aligned} \quad (2)$$

a_t is the command (off or on), α_t the fraction of the slot the element actually runs, $\bar{\alpha}_t$ the thermostat's cut-out, and Q_t^{out} the heat lost through the insulation and carried off by the hot water drawn (Appendix A.2). The last term charges a tank left short at the end the cost of reheating it.

Heat pump / HVAC (state: room temperature T_t ; electrical power P ; coefficient of performance COP; room capacitance C_{room} and envelope resistance R_{wall} ; comfort band $[T_t^{\text{lo}}, T_t^{\text{hi}}]$, which may change over the day):

$$\begin{aligned} \text{dynamics:} \quad & T_{t+1} = T_t + \frac{\Delta t}{C_{\text{room}}} \left(q(a_t) \alpha_t + \frac{T_t^{\text{out}} - T_t}{R_{\text{wall}}} \right), \\ \text{constraints:} \quad & a_t \in \{-1, 0, +1\} \text{ (cool, off, heat)}, \quad \alpha_t = \min\{|a_t|, \bar{\alpha}_t(T_t)\}, \quad p_t = P\alpha_t, \\ \text{private cost:} \quad & f(p) = \Delta t \sum_t \pi_K^{\text{hp}} \xi_{t+1}(T_{t+1}) + (1 \text{ h}) \pi_K^{\text{hp}} \xi_n(T_n), \end{aligned} \quad (3)$$

where $\xi_t(T) = [T - T_t^{\text{hi}}]^+ + [T_t^{\text{lo}} - T]^+$ is the excursion from the comfort band at time t , and the heat flow is $q(a) = P \text{ COP } a$ when cooling and $P(\text{COP} + 1) a$ when heating (Appendix A.3).

Electric vehicle (a battery that cannot discharge, whose charger is off or at least P^{min} , with a charging goal G_t , the energy wanted in store by slot t):

$$\text{as (1) with } \bar{P}_d = 0 \text{ and } p_t \in \{0\} \cup [P^{\text{min}}, \bar{P}_c], \quad f(p) = \pi_T (s_0 - s_n) + \kappa_{\text{goal}} \sum_t [G_t - s_t]^+. \quad (4)$$

Discomfort in currency. Restoring one kelvin to a thermal mass C costs $C\bar{c}/\text{COP}$ at the reference price $\bar{c}$ (the mean import price; $\text{COP} = 1$ for a resistive element). A kelvin-hour of discomfort is priced at κ times that,

$$\pi_K = \kappa C \bar{c} / \text{COP}, \quad \kappa = 3 \text{ by default}, \quad (5)$$

so that the bill and the discomfort are in the same unit and κ reads plainly: how many times the cost of fixing a drift one would pay to avoid it.

The water heater and the HVAC have on/off controls, and the EV's charger a minimum power, so their $\mathcal{P}_k$ are not convex. The battery, written with separate charge and discharge variables, is a linear programme (Appendix B.2).

1.3 The meter

The meter carries

$$z_t = d_t + \chi_t + \sum_{k=1}^m p_{k,t}, \quad (6)$$

positive for import, where $\chi_t \in [0, g_t]$ is PV output curtailed (discarded). Its cost in slot t is

$$m_t(z) = \Delta t \left(c_t^{\text{imp}} [z]^+ - c_t^{\text{exp}} [z]^- + c^{\text{br}} [z - \bar{z}^{\text{imp}}]^+ + c^{\text{br}} [-z - \bar{z}^{\text{exp}}]^+ \right). \quad (7)$$

The first two terms are the bill. The last two price energy beyond the import and export limits at $c^{\text{br}} = \kappa_g \max_t \max(|c_t^{\text{imp}}|, |c_t^{\text{exp}}|)$, with $\kappa_g = 10$: a price rather than a hard constraint, so that a plan exists even when a limit cannot be met; a breach then stays in the plan, priced and reported. (Without a limit, its term is zero.) Curtailing costs nothing and pays only when export is priced negatively or beyond the export limit.

When $c_t^{\text{exp}} \leq c_t^{\text{imp}}$, m_t is convex and piecewise linear: its slope is $c_t^{\text{exp}} - c^{\text{br}}$, c_t^{exp} , c_t^{imp} and $c_t^{\text{imp}} + c^{\text{br}}$ (per kWh) from export beyond the limit to import beyond it. We write $M_t(y) = \min_{0 \leq \chi \leq g_t} m_t(y + \chi)$ for the meter cost after the best curtailment.

1.4 The optimisation problem

$$\begin{aligned} \min_{p_1, \dots, p_m, \chi} \quad & J = \sum_{t=0}^{n-1} m_t(z_t) + \sum_{k=1}^m f_k(p_k) \\ \text{s.t.} \quad & z_t = d_t + \chi_t + \sum_k p_{k,t}, \quad 0 \leq \chi_t \leq g_t \quad (\text{meter balance, curtailment}) \\ & p_k \in \mathcal{P}_k, \quad k = 1, \dots, m \quad (\text{device constraints (1)–(4)}) \end{aligned}$$

(8)

Let J^* be its minimum. The devices share nothing but the meter. Were m_t linear — one price for both directions and no limits — J would be a sum of per-device terms and each device could plan alone against the tariff. The kink of m_t at $z = 0$, where the marginal price jumps from c_t^{exp} to c_t^{imp} , and the limits are the entire coupling: whether the next kWh a device draws is imported, or merely reduces an export, depends on what every other device does.

Problem (8) can be written as one mixed-integer linear programme (MILP), with binaries for the on/off controls and the charger minimum; we use one as a reference (Appendix C.1). As a home controller it has two drawbacks: it needs every device's model in one place and one formalism, and its solve time grows with the binaries — 15 to 39 s for one battery and one water heater over a day. The rest of these notes split (8) by device instead.

1.5 Why devices cannot simply take turns

The obvious split is to let each device plan against the others' current plans and repeat — all devices at once (a Jacobi iteration) or one at a time (Gauss–Seidel) [3]. Take two identical batteries and one hour of PV surplus. Updating together, each sees the surplus and plans to absorb all of it, so they import to absorb twice the surplus; next round each sees the expensive import and plans nothing; the round after repeats the first. Updating one at a time removes this cycle but can stop at a plan that no single device can improve and yet is not optimal: at the kink, an improvement may need several devices to move together while the meter stays balanced. A *coordinator* is needed: something that damps the overshoot and steers the plans to agree.

2 The device interface

Both coordinators talk to a device through the same interface, and neither sees its model. A device is an object that holds its model, its state and its solver, and answers queries; the coordinators differ only in which queries they send (Figure 1).

One query. Every query asks for the device's cheapest plan under a charge for its energy:

$$\mathcal{Q}_k(\pi^+, \pi^-, \delta, v, \rho) = \arg \min_{p \in \mathcal{P}_k} f_k(p) + \Delta t \sum_t \left(\pi_t^+ [\delta_t + p_t]^+ - \pi_t^- [\delta_t + p_t]^- \right) + \frac{\rho}{2} \|p - v\|^2, \quad (9)$$

with prices π^+ for drawing and π^- for supplying, a residual load δ under which the device's power is metered, and an optional pull of weight ρ toward a target plan v (with limits, the breach terms of (7) are added to the energy charge). The answer is the plan p , its private cost $f_k(p)$ and its state trajectory. Three settings of the parameters are all the coordinators use:

query	parameters of (9)	the device minimises	asked by
price response $\text{PR}_k(\pi)$	$\pi^+ = \pi^- = \pi, \delta = 0, \rho = 0$	$f_k(p) + \Delta t \pi^\top p$	Dantzig–Wolfe
proximal step $\text{PX}_k(v, \rho)$	$\pi^\pm = 0, \delta = 0$	$f_k(p) + \frac{\rho}{2} \ p - v\ ^2$	ADMM
best response $\text{BR}_k(\delta)$	$\pi^+ = c^{\text{imp}}, \pi^- = \max(c^{\text{exp}}, 0),$ limits, $\rho = 0$	$f_k(p) + \sum_t M_t(\delta_t + p_t)$	both

In BR_k , $\delta_t = d_t + \sum_{k' \neq k} p_{k',t}$ is the rest of the house, and pricing export at $\max(c^{\text{exp}}, 0)$ accounts for curtailment (Proposition 1). The queries charge the device for its energy in three ways: at a posted price, not at all but with a pull toward a target, or at the real tariff given everything else on the meter. A device needs the tariff only in BR, and never sees the other devices or the grid limits except through δ there.

The whole interface. Table 2 lists every call and which coordinator makes it. Everything except the last two rows is required of every device, and serves both coordinators unchanged: a device written once can be coordinated by either.

Because every call is an instance of (9), one solver routine serves both coordinators. In the package each device's DP is such a routine (`solve_battery`, `solve_water_heater`, `solve_hvac`), taking the two price vectors, the residual load, the limits, the target and the weight: Dantzig–Wolfe calls it with $(\pi, \pi, 0, \cdot, 0)$, ADMM with $(0, 0, 0, v, \rho)$, and both with the tariff and δ for BR. (Dantzig–Wolfe's best responses leave out the floor on the export price and the limits, which its master holds, and its extra offers use π where π lies outside the tariff; any runnable plan is a valid offer, and its polish keeps a plan only if J falls.) The battery's

call	returns	Dantzig–Wolfe	ADMM
query (9): $\text{PR}_k(\pi)$	plan, private cost, trajectory	offers, bound	—
query (9): $\text{PX}_k(v, \rho)$	plan, private cost, trajectory	—	every iteration
query (9): $\text{BR}_k(\delta)$	plan, private cost, trajectory	extra offers, polish	runnable plans, polish
cost of a given plan p	$f_k(p)$ and trajectory	scoring	scoring
baseline plan	the uncoordinated plan (thermostat, idle)	first offer	fallback
blend (if it modulates)	the plan a weighted blend of its plans produces	recovery	—
<i>optional</i> : linear model	its constraints and cost as an LP	held in the master	—
<i>optional</i> : policy	value function and policy over states	execution tier (Section 6)	

Table 2: The device interface, shared by both coordinators. A device whose model is a linear programme may hand its constraints to Dantzig–Wolfe, which then holds it exactly as variables (Section 3.2); the home battery does so by default, as does a water heater whose element modulates.

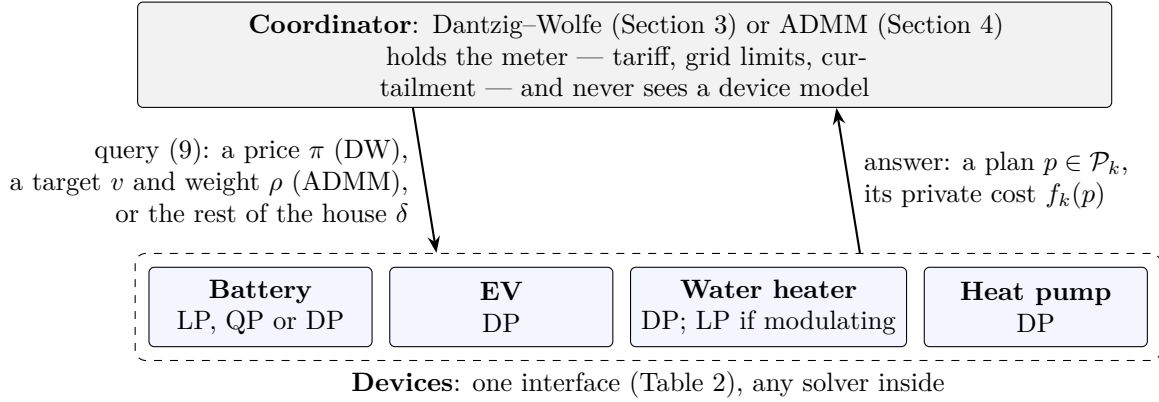


Figure 1: The architecture. Each device plans itself with the solver that suits it; the coordinator makes the plans agree at the meter.

LP step (`battery_prox`) implements PX alone. The coordinators call these routines directly and compute private costs from the returned trajectories; gathering the calls of Table 2 into one device class would be a refactor, not a change of method.

2.1 Solvers behind the interface

device	PR (Dantzig–Wolfe)	PX (ADMM)	BR (both)
battery	held in the master LP	DP, or its exact LP (option)	DP
EV	DP	DP	DP
water heater	DP; master LP if modulating	DP on a relaxed copy	DP
HVAC	DP	DP on a relaxed copy	DP

Table 3: The solvers the package uses behind each query. Relaxed copies are explained in Section 4.4.

DP. A backward recursion over the device’s own state, discretised (Appendix B.1). It suits a device with one or two state variables — its cost grows exponentially with their number, which

is why it is used per device and never for the whole house. It handles on/off controls, charger minima and goals simply by restricting the actions, is exact on its grid, and returns besides the plan a value function and a policy for every state, which the execution tier uses (Section 6).

LP and QP. The battery is a linear programme: charge, discharge and stored energy with linear constraints. Its price response is an LP; its proximal step a convex QP, which the package solves exactly by a small interior-point method (Appendix B.2).

Anything else. A MILP, a heuristic, a manufacturer’s black box: the only requirement is that answers be plans the device can run, with their true private costs. How well a solver answers decides what the coordinator can guarantee.

2.2 What exact answers buy

- **Dantzig–Wolfe** accepts any runnable plan as an offer, so an inexact solver only slows it. Its certificate (Section 3.3) needs each device’s *minimum value* in PR; a solver that may miss the best plan must report a lower bound on that value, or the certificate holds only up to the solver’s error. The DPs are exact on their grids, so here the certificate reads “within the gap, up to the DP grids”.
- **ADMM** converges to the optimal cost when every f_k and $\mathcal{P}_k$ is convex and every PX is exact [4]. On/off devices are not convex and DP steps are not exact, so with them ADMM is a heuristic (Section 4.3); for batteries alone with LP steps, it converges to the optimum.

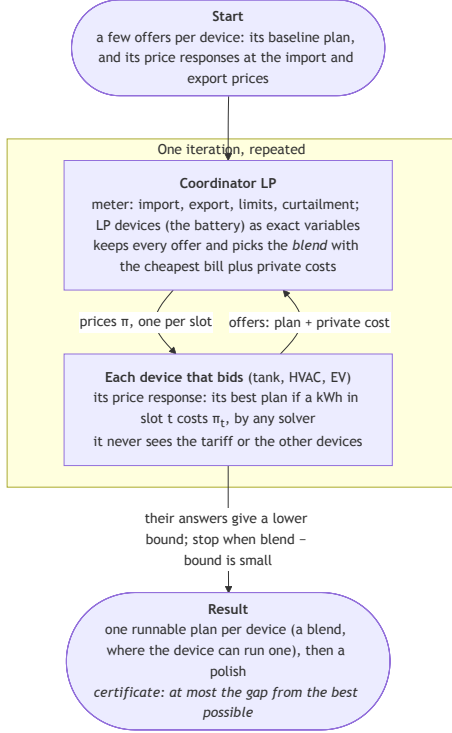
3 The first coordinator: Dantzig–Wolfe

3.1 The idea

Think of the house as a market with one shared good, energy at the meter. A coordinator posts a price per slot, π_t ; every device answers with its best plan at those prices, its price response, which we call an *offer*. The coordinator keeps every offer and chooses weights on them, summing to one per device, so that the meter balances at the least total cost. Its choice is a small linear programme whose duals are new prices; the devices answer again, and the loop repeats. In Dantzig and Wolfe’s words, the coordinator “works out a system of ‘prices’”, and “old offers are never forgotten by the central agency (unless using ‘current’ prices they are unprofitable)” [7, p. 102].

With the two batteries of Section 1.5, each offers to absorb the whole surplus. The coordinator takes *half* of each offer: the blend absorbs the surplus exactly and imports nothing, and the price it posts for that hour lies between the export and the import price. Offers are only added, so the coordinator’s value can only fall; nothing oscillates. Figure 2 (left) shows the loop.

Dantzig–Wolfe (Section 3)



ADMM (Section 4)

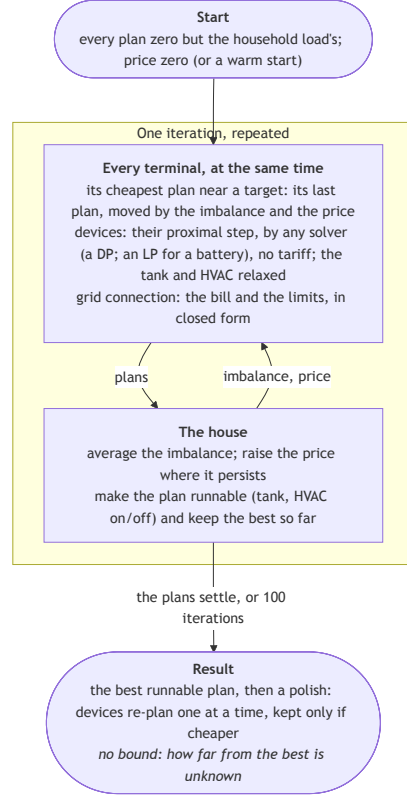


Figure 2: What goes round each coordinator’s loop. Sources: docs/figs/flow_*.mmd (Mermaid).

3.2 The coordinator’s problem

Let $\mathcal{K}$ be the devices that bid offers, each with a pool $\mathcal{J}_k$ of offers (p^{kj}, f_{kj}) , and $\mathcal{B}$ the batteries held as LP variables. The *master* LP is

$$\begin{aligned}
 \min \quad & \Delta t \sum_t \left(c_t^{\text{imp}} z_t^+ - c_t^{\text{exp}} z_t^- + (c_t^{\text{imp}} + c^{\text{br}}) o_t^+ - (c_t^{\text{exp}} - c^{\text{br}}) o_t^- \right) \\
 & + \sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} f_{kj} w_{kj} + \sum_{b \in \mathcal{B}} \pi_T (s_0^b - s_n^b) \\
 \text{s.t.} \quad & z_t^+ + o_t^+ - z_t^- - o_t^- = d_t + \chi_t + \sum_{k,j} w_{kj} p_t^{kj} + \sum_b (c_t^b - e_t^b) \quad \forall t \quad [\text{dual } \Delta t \pi_t] \quad (10) \\
 & \sum_{j \in \mathcal{J}_k} w_{kj} = 1 \quad \forall k \in \mathcal{K} \quad [\text{dual } \sigma_k] \\
 & s_{t+1}^b = s_t^b + \Delta t (\eta_c c_t^b - e_t^b / \eta_d) \quad \forall b, t \\
 & 0 \leq z_t^+ \leq \bar{z}^{\text{imp}}, \quad 0 \leq z_t^- \leq \bar{z}^{\text{exp}}, \quad o_t^\pm \geq 0, \quad 0 \leq \chi_t \leq g_t, \quad w_{kj} \geq 0, \\
 & 0 \leq c_t^b \leq \bar{P}_c, \quad 0 \leq e_t^b \leq \bar{P}_d, \quad \underline{S} \leq s_t^b \leq \bar{S}.
 \end{aligned}$$

The meter row is (6) with the flow split into import z^+ and export z^- within the limits and $o^\pm$ beyond them; the objective is $\sum_t m_t$ plus the offers’ and batteries’ private costs. A water heater held in the master adds its dynamics (2) at continuous duty in the same way. Three properties make the master easy to use:

- *The kink is exact without binaries.* Import and export are separate variables with separate prices; since $c_t^{\text{exp}} \leq c_t^{\text{imp}}$, importing and exporting in the same slot never pays. (A slot where export pays more than import is refused, and the planner falls back to ADMM.)

- *It is always feasible:* the breach variables absorb any imbalance, so any one offer per device starts it.
- *It is small:* a few hundred rows over a day.

What its prices mean. The dual of a row is how much the optimal cost rises if the row asks for one unit more. For the meter row of slot t this is π_t , the cost to the house of one more kWh drawn then. LP optimality says a variable in use breaks even at the row prices, so $\pi_t = c_t^{\text{imp}}$ where the house imports, c_t^{exp} where it exports, some value in $[c_t^{\text{exp}}, c_t^{\text{imp}}]$ where the meter is balanced, and $c_t^{\text{imp}} + c^{\text{br}}$ where it imports beyond the limit. The same rule on an offer in use gives $f_{kj} + \Delta t \pi^\top p^{kj} = \sigma_k$: σ_k is what device k 's current blend costs at the current prices.

3.3 Pricing and the certificate

Pricing. An offer p of device k with private cost f lowers the master's value if its *reduced cost* is negative,

$$f + \Delta t \pi^\top p - \sigma_k < 0, \quad (11)$$

that is, if at the current prices it is cheaper than the device's current blend. The offer with the most negative reduced cost is exactly the price response $\text{PR}_k(\pi)$ of Section 2. If no device has one, the master's blend is optimal.

The bound. For *any* prices $\pi \in \mathbb{R}^n$,

$$L(\pi) = \Delta t \pi^\top d + \sum_{k=1}^m \min_{p \in \mathcal{P}_k} (f_k(p) + \Delta t \pi^\top p) + \sum_t \min_{\substack{z \in \mathbb{R} \\ 0 \leq \chi \leq g_t}} (m_t(z) - \Delta t \pi_t (z - \chi)) \leq J^*. \quad (12)$$

The reasoning: at any prices, split the house into independent pieces — each device buying its energy at π , and the meter buying from the grid at the tariff and reselling at π — and let each piece do its best alone. Problem (8) adds only the rule that the pieces agree on every kWh, which cannot make anything cheaper. Formally, L is the Lagrangian dual function of (8) for its meter rows, and $L(\pi) \leq J^*$ is weak duality, which holds whether or not the devices are convex [5, Ch. 5]. Each device term is the value of its price response; the meter term is closed form (it is $-\infty$ unless π_t lies within the range of m_t 's slopes); a battery held in the master contributes its LP price response.

The certificate. The coordinator evaluates L at every price it asks at and keeps the best, L^{best} . For any plan of cost J ,

$$J - J^* \leq J - L^{\text{best}} =: \text{gap},$$

so no plan is cheaper than this one by more than the gap. This holds for any plan, including one made by ADMM. At the master's own prices, the master's value minus $L(\pi)$ equals $\sum_k (\sigma_k - \min_p (f_k(p) + \Delta t \pi^\top p))$: the gap closes exactly when no device has an offer with negative reduced cost. The bound needs each minimum exactly, or a lower bound on it; the code takes each device's minimum over its DP's answer and every offer it has made at that price, and records by how much the DP was undercut.

3.4 From a blend to a runnable plan

The master's answer is a blend: say 60% of one water-heater plan and 40% of another. A modulating device runs its blend through its own physics. For a battery the meter sees exactly the blended power, and because losses make stored energy concave in power, the store ends

with at least the blended energy. An on/off element cannot run a blend, nor can an HVAC (a blend could mix heating with cooling) or an EV whose charger has a minimum power. For those, the last step picks one offer per device from the pool by re-solving (10) with $w_{kj} \in \{0, 1\}$: a small MILP over a few dozen offers, solved by the package’s own branch and bound.

3.5 The algorithm

Algorithm 1 Dantzig–Wolfe planning cycle (`dw.coordinator`).

Require: forecasts $c^{\text{imp}}, c^{\text{exp}}, d, g$ and each device’s own; bidding devices $\mathcal{K}$, LP batteries $\mathcal{B}$; tolerance $\epsilon = 10^{-3}$; iteration cap (40)

- 1: **seed:** for each $k \in \mathcal{K}$, $\mathcal{J}_k \leftarrow \{\text{baseline plan}, \text{PR}_k(c^{\text{imp}}), \text{PR}_k(\max(c^{\text{exp}}, 0))\}$; $L^{\text{best}} \leftarrow -\infty$
- 2: **repeat**
- 3: solve the master (10) $\rightarrow$ value J^{LP} , weights w , LP-battery plans, duals π, σ
- 4: **for** each price π' in $\{\pi, \theta\pi^c + (1 - \theta)\pi\}$ **do** $\triangleright \pi^c$: price of the best bound so far
- 5: for each $k \in \mathcal{K}$: add $\text{PR}_k(\pi')$ to $\mathcal{J}_k$ (if new)
- 6: evaluate $L(\pi')$ by (12); if $L(\pi') > L^{\text{best}}$: $L^{\text{best}} \leftarrow L(\pi')$, $\pi^c \leftarrow \pi'$
- 7: **end for**
- 8: for each $k \in \mathcal{K}$: add $\text{BR}_k(\delta_k)$, $\delta_k = d + \text{the rest of the current blend}$ $\triangleright$ any runnable plan is a valid offer
- 9: **until** $J^{\text{LP}} - L^{\text{best}} \leq \epsilon \max(1, |J^{\text{LP}}|)$, or no new offer, or the cap
- 10: **recover** one runnable plan per device (Section 3.4)
- 11: **polish:** up to three sweeps; each k in turn takes BR_k against the others’ plans, kept only if J falls
- 12: **return** plan, its cost J , bound L^{best} , gap $J - L^{\text{best}}$, meter prices π

Algorithm 1 is the cycle. The second price in line 4, used from the second iteration on, is a smoothed one, pulled toward the prices that gave the best bound so far ($\theta = 0.5$, or 0.8 when a battery bids offers); any price gives valid offers and a valid bound, and the smoothed one damps the swings of the master’s duals. The best-response offers of line 7 help where a device should follow the meter’s balance point, which a single flat price reaches only by blending many offers. Grid limits and curtailment need no step of their own: they are part of the master. The testbed also recovers a plan at every iteration and keeps the best.

Cost. One iteration is one small LP plus, per bidding device, a price response at each of the two prices and one best response. The master is solved by an interior-point method in numpy (`dw.lpsolver`). A day at 15-minute slots takes 0.6–0.9 s on the Home Assistant demo site; a week-long evcc request (672 slots), 1.7–2.5 s.

4 The second coordinator: ADMM

ADMM coordinates the same devices without a linear programme. Each device responds to one shared price and a pull toward its last plan; the price rises wherever the house plans to draw more than it is supplied. It is the coordinator for a deployment without an LP solver, and the fallback when Dantzig–Wolfe cannot run. What it gives up is the certificate, though Dantzig–Wolfe’s bound certifies its plans too.

4.1 ADMM in brief

The alternating direction method of multipliers [4] solves $\min_{x,y} f(x) + g(y)$ subject to $Ax + By = c$, where f and g are each easy but their sum is not. With a scaled price u on the constraint,

it repeats three steps:

$$\begin{aligned}
x &\leftarrow \arg \min_x f(x) + \frac{\rho}{2} \|Ax + By - c + u\|^2, \\
y &\leftarrow \arg \min_y g(y) + \frac{\rho}{2} \|Ax + By - c + u\|^2, \\
u &\leftarrow u + (Ax + By - c).
\end{aligned} \tag{13}$$

The same square appears in both minimisations, and u is the running sum of past mismatches; ρu is the price on the constraint. In the form used below, each step is a *proximal operator*,

$$\text{prox}_{f,\rho}(v) = \arg \min_x f(x) + \frac{\rho}{2} \|x - v\|^2 : \tag{14}$$

minimise f , but stay near v ; ρ sets how strongly [17].

4.2 The coordinator's problem

Call everything connected to the meter a *terminal* $i = 1, \dots, N$, with a plan $p_i \in \mathbb{R}^n$ (positive when it draws) and a cost f_i :

terminal	plan	cost f_i	its proximal step
household load	$p = \ell$	0	returns ℓ
PV	$-g \leq p \leq 0$ (curtailment: $p > -g$)	0	clips v to $[-g, 0]$
grid connection	$p = -z$	$\sum_t m_t(-p_t)$	closed form, per slot
device k	$p \in \mathcal{P}_k$	$f_k(p)$	$\text{PX}_k(v, \rho)$

With these, (8) is the *exchange problem* [4, §7.3.2]

$$\min_{p_1, \dots, p_N} \sum_{i=1}^N f_i(p_i) \quad \text{s.t.} \quad \sum_{i=1}^N p_{i,t} = 0 \quad \forall t, \tag{15}$$

whose constraint is the meter balance (6). ADMM on (15), with the plans as x and a copy of them that must sum to zero as y , reduces to two lines [4, §7.3.2]:

$$p_i \leftarrow \text{prox}_{f_i, \rho}(p_i - \bar{p} - u) \quad \text{for every terminal } i, \text{ in parallel,} \tag{16}$$

$$u \leftarrow u + \bar{p}, \quad \bar{p}_t = \frac{1}{N} \sum_i p_{i,t}. \tag{17}$$

$\bar{p}$ is the house's imbalance per terminal and u its running sum. Where the house plans to draw more than it is supplied, u rises and every target $p_i - \bar{p} - u$ moves down: the devices draw less, the grid connection supplies more, at its price. No terminal sees another's plan; they meet only in $\bar{p}$ and u . Kraning, Chu, Lavaei and Boyd call this *proximal message passing* [12]. The grid connection's step is closed form because m_t is piecewise linear: on each linear piece, the minimiser of cost plus square is the stationary point clipped to the piece; the step takes the cheapest of these. The tariff and the limits live in this one step, and no device sees them.

With the two batteries of Section 1.5: when both plan to charge into the same surplus, $\bar{p} > 0$ in those slots, u rises there, and both targets move down together, while the square keeps either from jumping.

Stopping and ρ . The residuals are the imbalance $r_p = \sqrt{N} \|\bar{p}\|$ and the movement $r_d = \rho \|(p - \bar{p}) - (p - \bar{p})^{\text{prev}}\|$ (over all terminals and slots); the loop stops when both are below $\epsilon^{\text{abs}} \sqrt{Nn}$, with $\epsilon^{\text{abs}} = 10^{-3}$ kW. ρ follows the rule of [12]: with $\omega = \rho r_p / r_d - 1$, $\rho \leftarrow \rho \exp(0.01 \omega + 0.01 (\omega - \omega^{\text{prev}}))$, u rescaled so that ρu is unchanged, and ρ held after 200 iterations.

4.3 When it converges, and when it does not

For closed convex costs with a saddle point, and exact proximal steps, the imbalance vanishes, the total cost converges to its minimum and the prices to optimal prices [4, §3.2]. The proof needs both. Its key step infers from “ p minimises f plus a square” that “ p minimises f plus a linear term”, which holds only for convex f [4, App. A]; and errors in the steps are tolerated only if summable [4, §3.4.4]. A home breaks both:

- *On/off devices are not convex.* Under the square, an on/off element can only stay put or jump by its full power, however the price moves.
- *DP steps are neither exact nor convex.* On its grid a battery’s plan moves by a whole power step or not at all, so the error never shrinks. One battery alone still converges. Several batteries facing one price reach their switching points together: they move at once, the same way, overshoot the balance, and the price turns them all back. With eight batteries the imbalance stalls near 2.5 against a threshold of 0.04; a finer grid shrinks the stall in proportion but does not remove it (Appendix C.2).

Exact LP steps for the batteries remove the second: each battery’s plan then moves continuously with the price. On a site of batteries alone every cost is convex and every step exact, and ADMM converges to the optimum, to within what its stopping test asks — 0.049 above the optimum at the default tolerance, 0.001 at a hundredth of it (Appendix C.2). With on/off devices present it remains a heuristic, so the implementation keeps the best runnable plan it sees, not the last iterate.

The device steps in (16) are simultaneous by design: the plans form one block, and with u held the problem separates across terminals. Letting each device see its predecessor’s fresh plan would be ADMM with one block per device, which can diverge even on convex problems [6].

4.4 The algorithm

Algorithm 2 ADMM planning cycle, as implemented (`admm.coordinator`).

Require: forecasts; terminals: load, PV, grid connection, and each device, the water heater and HVAC as relaxed copies; $\rho_0 = 0.1$; budget 100 iterations

- 1: $p_{\text{load}} \leftarrow \ell$, every other $p_i \leftarrow 0$; $u \leftarrow 0$; $\rho \leftarrow \rho_0$ ▷ or a warm start
 - 2: **for** iteration = 1, ..., 100 **do**
 - 3: targets: $v_i \leftarrow p_i - \bar{p} - u$ for every terminal i
 - 4: steps, in parallel: load ℓ ; PV $\min(\max(v, -g), 0)$; grid connection in closed form; each device $p_k \leftarrow \text{PX}_k(v_k, \rho)$
 - 5: $\bar{p}_t \leftarrow \frac{1}{N} \sum_i p_{i,t}$; $u \leftarrow u + \bar{p}$; residuals r_p, r_d
 - 6: **runnable plan:** each relaxed device takes its real device’s BR against the others’ plans; score by J ; keep if the best so far
 - 7: stop if $r_p \leq \epsilon^{\text{abs}} \sqrt{Nn}$ and $r_d \leq \epsilon^{\text{abs}} \sqrt{Nn}$
 - 8: update ρ (Section 4.2) during the first 200 iterations
 - 9: **end for**
 - 10: **baseline fallback:** a device takes its baseline plan if that lowers J
 - 11: **polish:** up to three sweeps; each device in turn takes its BR against the others’ plans, kept only if J falls
 - 12: **return** the plan (and the pricing re-solve of Section 6)
-

Algorithm 2 departs from proximal message passing as published [12] in five ways, each measured in Appendix C.2:

1. *Device steps* are the devices' DPs by default (inexact), a battery's optionally its exact LP. No solver is needed by default.
2. *On/off devices iterate as relaxed copies* that may run twelfths of a slot, close to the convex relaxations the paper suggests for non-convex devices. Without them the plans end three times as far from the bound.
3. *A runnable plan every iteration* (line 6), and the best one kept: the copies' plans cannot be run, and the iterates need not settle.
4. *Baseline fallback and polish* at the end; neither can raise J . The polish carries most of the quality: on twelve test sites the iterations' best plan lies 2.0 above the Dantzig–Wolfe bound, 0.42 after polishing.
5. *Tuning*: while iterating, the bill's kink is rounded over $|z| < 0.25$ kW (scoring uses the exact bill), which makes the price at a balanced meter unique; and the budget is 100 iterations (300 come only 0.02 closer, at twice the time).

Warm starts. A solve can start from where an earlier one stood at its best plan — every terminal's plan, u and ρ — when the devices and horizon match; for the next planning cycle the state is shifted by the slots that have passed. Each LP battery step likewise starts from that battery's previous solution. A warm start reaches a good plan in about a quarter of the iterations, though not a better final plan (Appendix C.2).

Cost and quality. On 72 test sites ADMM's plans come within 0.51 of the Dantzig–Wolfe bound on average (Dantzig–Wolfe's own: 0.10), at about ten times its time. Against a MILP over the same model it captures 98.1–99.7% of the available savings, Dantzig–Wolfe 99.5–99.7% (Figure 7).

5 What a plan comes with: prices

5.1 The meter price

Dantzig–Wolfe's meter dual π_t (Section 3.2) is the cost to the house of one more kWh drawn in slot t , counting every device and every limit. A flexible load deciding when to run can compare its own value per kWh with it. It is fixed until the next plan.

5.2 The value of stored energy

Let λ_t be the marginal value of stored energy: how much the best cost of the rest of the horizon falls per extra kWh in store at the start of slot t . Where it comes from depends on the battery's solver. An LP gives it as the dual of the storage balance row in (1), along the planned trajectory. A DP gives it as the slope of its value function, $\lambda_t(s) = \partial V_t / \partial s$, at *every* state s (Appendix B.1), which is why the execution tier uses a DP (Section 6). On the test fixtures the DP's λ lies within 0.007–0.032 cur/kWh of the LP's dual, the error of its grid.

Reservation prices. Consider a battery facing a price c_t per kWh at the meter. The optimality conditions of its LP, with λ_{t+1} the dual of the balance row for slot t , say that it charges only if $c_t \leq \eta_c \lambda_{t+1}$ (a kWh bought stores η_c of it), discharges only if $c_t \geq \lambda_{t+1} / \eta_d$ (a kWh sold drains $1/\eta_d$), and holds in between, with equality wherever its power lies strictly inside its limits. So, with λ the value of energy stored during the slot,

$$\text{import while } c_t < \eta_c \lambda, \quad \text{export while } c_t > \lambda / \eta_d, \quad \text{hold in between.} \quad (18)$$

The band's width $\lambda(1/\eta_d - \eta_c)$ is the round-trip loss; for a lossless store it vanishes. The price c_t is the *prevailing* one: c_t^{imp} while the house imports, c_t^{exp} while it exports, since a discharge that displaces load saves the import price. On a flat tariff with $c^{\text{exp}} = 0.08$, $c^{\text{imp}} = 0.30$ and $\eta_c = \eta_d = 0.9$, the LP's dual attains $0.08/0.9 = 0.089$ where the battery charges from surplus and $0.30 \times 0.9 = 0.27$ where it discharges into the load. Figure 3 shows λ and the band on a dynamic tariff.

The marginal cost of consumption. A flexible load asks what one more kWh of demand costs. It is met by the cheapest route,

$$\pi_t^{\text{meter}}(s) = \min \{ c_t^{\text{imp}}, \lambda_t(s)/\eta_d, c_t^{\text{exp}} \text{ if } \delta_t < 0 \} : \quad (19)$$

buying, drawing on the store, or forgoing export when there is a surplus. With storage it is often far below c_t^{imp} . It holds the rest of the plan fixed where a true dual would let it re-optimize; against an LP finite difference its mean error is 0.0007–0.0053 cur/kWh, worst 0.043.

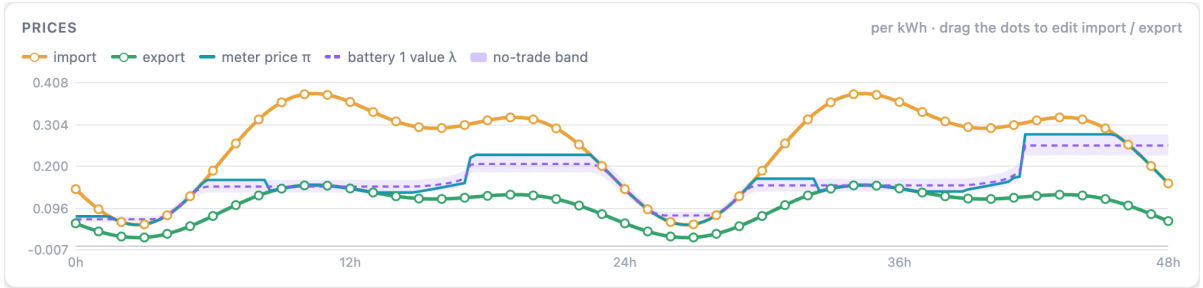


Figure 3: One battery with the water heater and HVAC, a two-day dynamic tariff, planned by Dantzig–Wolfe. λ (dashed) sits mostly between the export price (green) and the import price (orange), close to the plan's meter price π (teal), inside the shaded band (18): import below its lower edge, export above its upper edge, hold in between. In the cheapest night hours the import price drops below the band and the battery charges. λ is flat for hours and steps where the battery commits: stored energy is worth what it will fetch later, not what it costs now.

5.3 Prices when devices are on/off

With on/off controls the problem is a MILP, and no single price vector need support its optimum. Prices still exist by established constructions: the duals of the LP left once the integer decisions are fixed [15], and convex-hull (minimum-uplift) prices from the Lagrangian dual [10]. Both give a price at the optimal plan. A device acting between plans needs λ at the state it is actually in, which the DP's value function supplies from a solve performed anyway.

6 From plan to action

A home changes at two speeds, and the package has one tier for each:

- **Planning**, when the forecasts change (every 5–15 minutes): a coordinator plans the horizon (Algorithm 1 by default, Algorithm 2 as the fallback). Then the *pricing re-solve*: the first battery's DP at the tariff, against the residual load of the plan, $\delta = d + \sum_{k \neq \text{battery}} p_k$, gives a value function $V_t(s)$ and a policy $\text{POL}_t(s)$ free of any coordinator's artefacts (the proximal term, the master's duals).
- **Execution**, on every sensor update: look up the action $\text{POL}_t(s)$ and $\lambda_t(s)$ at the *measured* state s , by interpolation on the stored grid, and publish λ , the band (18), π^{meter} and the setpoint. No solver runs.

A battery that has drifted from its forecast thus gets a correct instruction at once, from the state it is in. This is the one place where the choice of solver matters beyond the plan: the tier needs a policy over states, which a DP returns; with an LP the tier would re-solve from each measured state (8–9 ms for a battery), making its cadence the solver’s.

Home Assistant receives prices and setpoints as ordinary sensors, so any automation can compare them with a number:

sensor	meaning	source
hems_lambda	value of a stored kWh, λ	execution
hems_import_below, _export_above	the band (18)	execution
hems_worth_running	π^{meter} from the battery’s state (19)	execution
hems_battery_action	setpoint now	execution
hems_meter_price	π_t of the plan: every device and limit	plan
hems_plan_gap	the certificate: at most this far from the best	plan

Figure 4 shows them live.

evcc calls the planner over HTTP with its own request format (**evcc**). The home battery and each charging point arrive as batteries: the home battery is held in the master, and each EV bids offers from its DP, which respects its charger minimum and goal. **evcc**’s grid limits become limits in the master, and the response carries the certificate. A request where export pays more than import in some slot, or an LP the solver gives up on, is planned by ADMM instead, and the response says so.

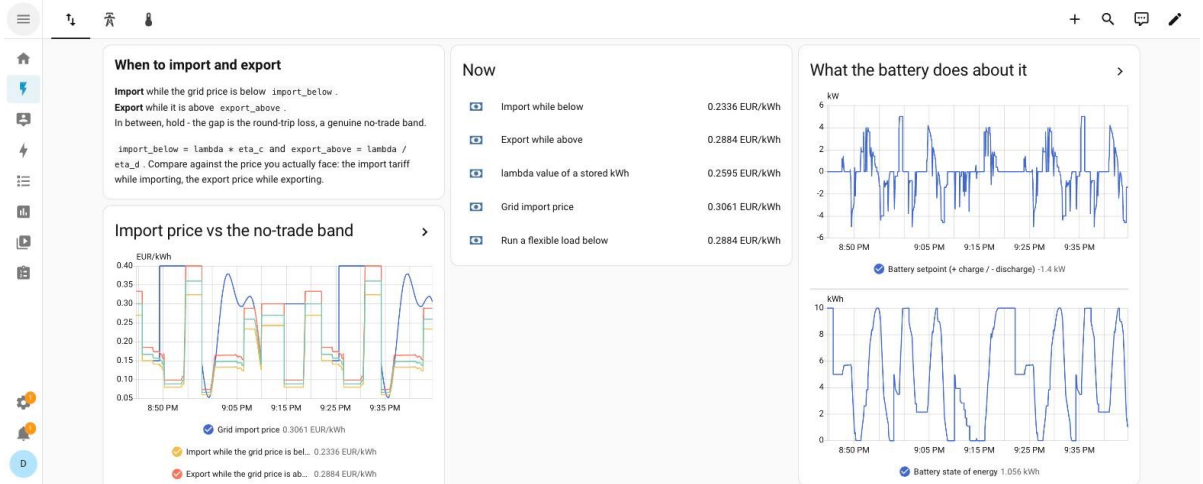


Figure 4: λ and the band streamed into Home Assistant as sensors, sampled live: $\lambda = 0.2595$, band from $0.2336 = \lambda\eta_c$ to $0.2884 = \lambda/\eta_d$ at $\eta_c = \eta_d = 0.9$. The import price, 0.3061, is above the upper edge, and the setpoint is correspondingly negative (−1.4 kW, discharging): the policy doing what the band says, with no logic in the dashboard beyond comparing two numbers.

7 Who saves what

The house pays one bill, and the plan lowers it; what did the solar and each device contribute? The bill does not split by device on its own: whether a kWh a device draws is imported or merely displaces an export depends on everything else on the meter. The package splits the saving as follows (**dw.attribution**).

Reimburse, then share. Compare the plan with a *baseline*: no PV, the thermal devices on their thermostats, the batteries idle. Device k is first paid its private-cost change $\Delta f_k = f_k(p_k) - f_k(p_k^0)$, so no device ends worse off than at its baseline. The rest, the total saving T (bill saving minus all private-cost changes), is shared among the solar and the devices.

Among the devices: Aumann–Shapley. As the devices move together from their baseline plans p^0 to the plan p , slot t ’s flow moves along a line from z_t^0 to z_t^1 . Each kWh a device shifts is priced at the average price the house meets along the way [1],

$$\hat{c}_t = \frac{M_t(z_t^1) - M_t(z_t^0)}{\Delta t (z_t^1 - z_t^0)}, \quad \text{gain}_k = -\Delta t \sum_t \hat{c}_t (p_{k,t} - p_{k,t}^0) - \Delta f_k,$$

with $\hat{c}_t$ the slope of $M_t/\Delta t$ where the flow does not change. No re-optimisation is needed, and the gains add up to the devices’ saving exactly.

Solar and the devices: two orders of arrival. Pricing slot by slot misses what storage does with PV — turn a midday kWh into an evening one — so solar is not simply one more player. Instead, average two orders: *PV first*, where solar earns what PV saves on its own, v_{PV} , and the devices share the rest; and *devices first*, where the devices share what a plan with no PV saves, v_{D} , and solar earns the rest. Solar’s gain is $\frac{1}{2}(v_{\text{PV}} + T - v_{\text{D}})$. This is the Owen value [16] for two groups, with Aumann–Shapley inside the group of devices; it needs one extra plan (the devices with no PV) and adds up to T exactly. Table 4 is an example. Against the Shapley value [21], which needs 2^{m+1} plans, the rule came within 0.46 on six test cases, and within 0.13 for the batteries (Appendix C.4).

player	bill before	bill after	change	private-cost change	net gain
household load	6.462	6.462	0	0	0
solar	0	−7.481	−7.481	0	7.481
battery 1	0	−1.604	−1.604	0.088	1.516
battery 2	0	−1.677	−1.677	−0.120	1.797
water heater	5.850	5.370	−0.480	−2.239	2.720
HVAC	0.356	0.147	−0.210	0.046	0.163
house	12.668	1.216	−11.452	−2.225	13.677

Table 4: Who saves what: two batteries (10 and 12 kWh), on/off water heater, HVAC, 5 kW of PV, a 7 kW import limit, day/night tariff, planned by Dantzig–Wolfe; cur/day (`bench/der_attribution.py`). A player’s bill before is its share of the baseline bill; bill after = bill before − gain $-\Delta f$.

8 The two coordinators compared

Table 5 runs both coordinators on the two integration paths and scores both plans with the same objective J . “Above bound” is J minus the Dantzig–Wolfe lower bound, a guaranteed limit on how much better any plan could be; it applies to ADMM’s plans too.

Dantzig–Wolfe’s plans are cheaper everywhere, by 0.03–0.30 per day, and it is an order of magnitude faster. Its certificate is tight on the Home Assistant site and looser under a binding limit, where the EV must fit around the limit in on/off steps of at least 4.14 kW while the bound assumes it could blend its offers.

case	Dantzig–Wolfe			ADMM		
	cost	above bound	time	cost	above bound	time
<i>Home Assistant demo site: battery, on/off water heater, HVAC; 24 h</i>						
flat tariff	3.577	0.075	0.6 s	3.603	0.102	8.2 s
day/night	1.961	0.052	0.9 s	2.076	0.167	13.7 s
dynamic	−0.189	0.083	0.8 s	−0.123	0.149	15.1 s
<i>evcc: home battery + EV (4.14 kW minimum, 40 kWh by 07:00); 24 h</i>						
no binding limit	−2.557	0.010	0.5 s	−2.255	0.312	4.0 s
import ≤ 7 kW	−2.557	0.010	0.5 s	−2.255	0.312	4.0 s
import ≤ 5 kW	−2.426	0.141	0.6 s	−2.255	0.312	4.1 s

Table 5: Cost is J in cur/day (`bench/run_planners.py`; times vary by a few tens of percent between runs, and the master’s prices are often not unique, so a small change in how it is solved can move a row by up to 0.07). On the Home Assistant site Dantzig–Wolfe captures 98–99% of the savings available over the baseline, ADMM 97–98%. Both met the EV’s goal and charger minimum; neither breached a limit.

	Dantzig–Wolfe	ADMM
guarantee	a gap on every plan, up to the solvers’ exactness	none of its own; converges for convex devices with exact steps
device queries	PR (and BR for extra offers and polish)	PX (and BR for runnable plans and polish)
on/off devices	bid runnable plans; one chosen by a small MILP	relaxed while iterating, made runnable every iteration
grid limits	rows of the master: met, or a priced breach	in the grid connection’s step: met, or a small priced breach
tuning	a tolerance	ρ and an iteration budget
speed (one day)	~ 1 s	~ 10 s
fails when	export pays more than import (falls back to ADMM)	quietly: a mediocre plan is not flagged

Recommendation. Dantzig–Wolfe is the default. ADMM is the fallback, for the tariff Dantzig–Wolfe cannot take, and the choice where no LP solver is wanted.

9 What is guaranteed, and what is not

Guaranteed.

- Dantzig–Wolfe’s gap bounds how far any plan is from J^* , up to the exactness of the price responses (the DP grids).
- On a site whose devices are all convex, with exact steps, ADMM converges to the optimum (batteries alone, with LP steps).
- Every state bound holds by construction, with no clipping: the battery’s admissible power keeps s in $[\underline{S}, \bar{S}]$, and the water heater’s outflow cap and cut-out keep T within its range (Appendix A).
- Curtailing after the device solves is exact when devices are priced at $\max(c^{\text{exp}}, 0)$ (Proposition 1).
- The reservation prices (18) follow from the battery’s optimality conditions.

Not guaranteed.

- **ADMM with on/off devices or DP steps** is a heuristic with no certificate of its own; Dantzig–Wolfe’s bound certifies its plans.
- λ **is conditional**: solved against the other devices’ planned powers, it is a marginal value given those plans, not a system-wide dual. Two batteries on one meter need not report the same λ .
- λ **carries the grid’s error** (0.007–0.032 cur/kWh); a ± 0.04 band is published with it, and decisions inside it should not be automated.
- **A plan may carry a small priced breach**: Dantzig–Wolfe meets a limit or reports the breach; ADMM may leave a few watts over it where a device’s grid has no step that lands on the headroom (at most 5 W on the test sites).
- **The execution tier follows the storage state, not the meter**. A deviation that moves the battery’s state is tracked; one that does not — a large load switching on while the battery idles — changes nothing until the next plan. Limits are respected in the plan, not enforced live. Clipping setpoints against the measured meter is an option (`policy.clamp`); with several batteries it must allocate the headroom sequentially (`clamp_fleet`), or it double-counts it as in Section 1.5. Only the first battery gets a live policy; others follow their planned powers until the next plan.
- **Model simplifications**. The water heater is one well-mixed node (a real tank stratifies, so the model understates deliverable comfort during a reheat); its surroundings and mains temperatures are constants; transitions are explicit Euler steps; and treating the element as on/off is a modelling choice, since over 15 minutes a thermostat cycles many times.

10 Related work

The residential scheduling problem in its modern form — a horizon, a price forecast, appliance models, energy cost traded against discomfort — dates to Mohsenian-Rad and Leon-Garcia [13]; its multi-household version coordinates homes through a shared price [14]. The literature mostly solves it as one MILP [11]. The first coordinator here is the decomposition principle of Dantzig and Wolfe [7], in the form the paper calls a generalised programme: offers may come from any set, so the devices need not be linear programmes. The second is ADMM [4] in exchange form, which for networks of energy devices is proximal message passing [12]; ADMM has also coordinated electric-vehicle fleets [20]. With discrete devices ADMM is a heuristic: its convergence results for nonconvex problems rest on smoothness an on/off element lacks [23], and “run, keep the best point, polish” is the recipe used for nonconvex sets generally [8]. The marginal value of stored energy comes from hydro scheduling, where the slope of a future-cost function prices stored water [18, 19]; for end-user storage the optimal policy is of threshold type, with thresholds moving with price [22], which is the band (18). Prices under nonconvexity follow the electricity-market literature [15, 10], and the savings split cooperative game theory [21, 1, 16]. What is specific here is the interface: both coordinators run the same devices through the same three queries, with the solver behind each chosen per device.

References

- [1] R. J. Aumann and L. S. Shapley. *Values of Non-Atomic Games*. Princeton University Press, 1974.
- [2] D. P. Bertsekas. *Dynamic Programming and Optimal Control*, Vol. I, 4th ed. Athena Scientific, 2017. The finite-horizon recursion of Appendix B.1.

- [3] D. P. Bertsekas and J. N. Tsitsiklis. *Parallel and Distributed Computation: Numerical Methods*. Prentice Hall, 1989. Jacobi and Gauss–Seidel block iterations and their convergence.
- [4] S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein. *Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers*. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011. §3 the algorithm, its scaled form and its convergence; §3.4.1 varying ρ ; §7.3.2 the exchange problem.
- [5] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. Ch. 5: Lagrangian duality and weak duality.
- [6] C. Chen, B. He, Y. Ye and X. Yuan. The direct extension of ADMM for multi-block convex minimization problems is not necessarily convergent. *Mathematical Programming*, 155(1–2):57–79, 2016.
- [7] G. B. Dantzig and P. Wolfe. Decomposition Principle for Linear Programs. *Operations Research*, 8(1):101–111, 1960.
- [8] S. Diamond, R. Takapoui and S. Boyd. A General System for Heuristic Minimization of Convex Functions over Non-Convex Sets. *Optimization Methods and Software*, 33(1), 2018.
- [9] T. Goldstein, B. O’Donoghue, S. Setzer and R. Baraniuk. Fast Alternating Direction Optimization Methods. *SIAM Journal on Imaging Sciences*, 7(3), 2014.
- [10] P. R. Gribik, W. W. Hogan and S. L. Pope. Market-Clearing Electricity Prices and Energy Uplift. Working paper, Harvard University, 2007.
- [11] I. Gomes, K. Bot, M. G. Ruano and A. Ruano. Recent Techniques Used in Home Energy Management Systems: A Review. *Energies*, 15(8):2866, 2022.
- [12] M. Kraning, E. Chu, J. Lavaei and S. Boyd. Dynamic Network Energy Management via Proximal Message Passing. *Foundations and Trends in Optimization*, 1(2):70–122, 2013.
- [13] A.-H. Mohsenian-Rad and A. Leon-Garcia. Optimal Residential Load Control With Price Prediction in Real-Time Electricity Pricing Environments. *IEEE Transactions on Smart Grid*, 1(2):120–133, 2010.
- [14] A.-H. Mohsenian-Rad, V. W. S. Wong, J. Jatskevich, R. Schober and A. Leon-Garcia. Autonomous Demand-Side Management Based on Game-Theoretic Energy Consumption Scheduling for the Future Smart Grid. *IEEE Transactions on Smart Grid*, 1(3):320–331, 2010.
- [15] R. P. O’Neill, P. M. Sotkiewicz, B. F. Hobbs, M. H. Rothkopf and W. R. Stewart Jr. Efficient market-clearing prices in markets with nonconvexities. *European Journal of Operational Research*, 164(1):269–285, 2005.
- [16] G. Owen. Values of Games with a Priori Unions. In R. Henn and O. Moeschlin (eds.), *Mathematical Economics and Game Theory*, Lecture Notes in Economics and Mathematical Systems 141, pp. 76–88. Springer, 1977.
- [17] N. Parikh and S. Boyd. Proximal Algorithms. *Foundations and Trends in Optimization*, 1(3):127–239, 2014.
- [18] M. V. F. Pereira and L. M. V. G. Pinto. Multi-stage stochastic optimization applied to energy planning. *Mathematical Programming*, 52(1–3):359–375, 1991.

- [19] A. B. Philpott. On the Marginal Value of Water for Hydroelectricity. In *Advances and Trends in Optimization with Engineering Applications*, SIAM, 2017.
- [20] J. Rivera, P. Wolfrum, S. Hirche, C. Goebel and H.-A. Jacobsen. Alternating Direction Method of Multipliers for Decentralized Electric Vehicle Charging Control. *IEEE Conference on Decision and Control*, 2013.
- [21] L. S. Shapley. A Value for n -Person Games. In H. W. Kuhn and A. W. Tucker (eds.), *Contributions to the Theory of Games II*, Annals of Mathematics Studies 28, pp. 307–317. Princeton University Press, 1953.
- [22] P. M. van de Ven, N. Hegde, L. Massoulié and T. Salonidis. Optimal control of end-user energy storage. arXiv:1203.1891, 2012.
- [23] Y. Wang, W. Yin and J. Zeng. Global Convergence of ADMM in Nonconvex Nonsmooth Optimization. *Journal of Scientific Computing*, 78, 2019.

A Device models

A.1 Battery

The balance (1) holds exactly, with no saturation, because the admissible power already respects the store’s bounds:

$$p_t \in \left[-\bar{P}_d, \bar{P}_c \right] \cap \left[-\frac{(s_t - \underline{S})\eta_d}{\Delta t}, \frac{\bar{S} - s_t}{\eta_c \Delta t} \right]. \quad (20)$$

Filling the headroom $\bar{S} - s$ needs meter power $(\bar{S} - s)/(\eta_c \Delta t)$, more than $(\bar{S} - s)/\Delta t$; drawing down to the reserve allows only $(s - \underline{S})\eta_d/\Delta t$. The admissible power shrinks as either bound approaches, so s_t never leaves $[\underline{S}, \bar{S}]$. A reserve above the starting state is lowered to it: the battery may charge but not discharge further. The terminal price π_T is linear so that $\partial f/\partial s_n$ stays a price at the horizon’s edge; the library uses the lowest import price, and the testbed the mean, which stops a battery emptying itself by the end of the day at the cost of the opposite lean.

A.2 Water heater

A single well-mixed node: uniform temperature T , constant density ρ_w and specific heat c_p , and every litre drawn replaced by mains water at T^{in} . With $C = \rho_w c_p V$, element power P , insulation resistance R to the air around the tank at T^{surr} , and a draw of W_t m³/h, conservation of energy gives

$$C \dot{T} = P\alpha - \frac{T - T^{\text{surr}}}{R} - \rho_w c_p W (T - T^{\text{in}}). \quad (21)$$

The two losses reference different temperatures, as they must: the insulation leaks to the room, the tap is refilled from the main, and a tank run flat settles at mains temperature. A draw forecast is given as the heat it removes at the setpoint, $D_t = \rho_w c_p W_t (T^* - T^{\text{in}})$ [kW]. With $\beta = 1/(RC)$, $\gamma_t = D_t/(C(T^* - T^{\text{in}}))$ and the equilibrium $T_t^\infty = (\beta T^{\text{surr}} + \gamma_t T^{\text{in}})/(\beta + \gamma_t)$, the tank relaxes toward T_t^∞ at rate $\beta + \gamma_t$. Over a slot the heat leaving is

$$Q_t^{\text{out}}(T) = C \min\{\beta + \gamma_t, 1/\Delta t\} (T - T_t^\infty), \quad (22)$$

and the state moves by (2). The cap lets the tank relax at most to T_t^∞ in one slot, so with the element off T_{t+1} lies between T_t^∞ and T_t , and $\min_t T_t^\infty$ is an exact floor for the state grid. (At the default parameters, $C = 0.174$ kWh/K, $R = 80$ K/kW, a peak draw of 4 kW, $(\beta + \gamma_t)\Delta t = 0.16$, so the cap is inactive.) The thermostat cut-out $\bar{T}$ bounds the *action*:

$$\bar{\alpha}_t(T) = \min \left\{ 1, \frac{(\bar{T} - T) C/\Delta t + Q_t^{\text{out}}(T)}{P} \right\}, \quad (23)$$

and a commanded on realises $\alpha_t = \bar{\alpha}_t(T_t)$ in the slot that reaches the cut-out. No action carries the tank past $\bar{T}$; over draws from $1\times$ to $200\times$ the fixture it stays within $[15, 75]$ °C with nothing clipped.

A.3 Heat pump / HVAC

The same single-node balance with the heat pump in place of the element and no draw: the room exchanges heat with the outdoor air through the envelope and by infiltration, which combine into one resistance R_{wall} , giving (3). Heating delivers COP + 1 times the electrical power (the compressor work also ends up in the room), cooling removes COP times it. The duty is capped as in (23), at the fraction that reaches the top of the modelled range when heating or its bottom when cooling. The end term prices the final excursion from the comfort band for one hour.

A.4 Electric vehicle

evcc describes a charging point as a battery that only charges, with a per-slot charge floor, a charger minimum (off or at least $P^{\min}$) and an energy goal. The goal is soft, priced at κ_{goal} per kWh of shortfall per slot (100 by default), so an unreachable goal degrades to “as close as possible”. A DP enforces the charger minimum by deleting actions; an LP would need one binary per slot, which is why an EV bids offers rather than sitting in the master.

B Device solvers

B.1 Dynamic programming

A device with state x on a grid $\mathcal{X}$ and actions a on a grid $\mathcal{A}(x)$ (restricted to what the state allows, as in (20) and (23)), transition $x_{t+1} = F_t(x_t, a_t)$ and power $p(a)$ answers a query by the backward recursion

$$V_n(x) = -\Phi(x), \quad V_t(x) = \max_{a \in \mathcal{A}(x)} \left\{ -\text{cost}_t(x, a) + V_{t+1}(F_t(x, a)) \right\}, \quad (24)$$

with V_{t+1} interpolated between grid points, followed by a forward rollout from the measured state [2]. Φ is the end term of f , and the stage cost is the device’s own (discomfort, goal shortfall) plus the query’s charge for its energy:

$$\text{PR} : \Delta t \pi_t p(a); \quad \text{PX} : \frac{\rho}{2} (p(a) - v_t)^2; \quad \text{BR} : m_t (\delta_t + p(a)) \text{ with } \max(c_t^{\text{exp}}, 0) \text{ for } c_t^{\text{exp}}.$$

The cost is $O(n|\mathcal{X}||\mathcal{A}|)$ per solve. Defaults: battery 200 energy levels and 81 power levels (the testbed, 50 levels); water heater 68 temperatures; HVAC 40. The relaxed copies ADMM iterates allow 13 duty levels. Besides the plan, a solve returns $V_t(x)$ and the policy $\text{POL}_t(x)$ at every grid state; $\partial V_t / \partial s$ is λ .

B.2 The battery as an LP, and its proximal step

With charge $c_t \in [0, \bar{P}_c]$, discharge $e_t \in [0, \bar{P}_d]$, stored energy $s_t \in [\underline{S}, \bar{S}]$ and $p = c - e$, the battery is the LP

$$s_{t+1} = s_t + \Delta t (\eta_c c_t - e_t / \eta_d), \quad \text{PR} : \min \Delta t \pi^\top (c - e) - \pi_T s_n, \quad (25)$$

which the master (10) holds as variables. Charging and discharging in one slot is allowed; it only wastes energy, so an optimum does it only where wasting energy costs nothing. The proximal step replaces the objective by $\frac{\rho}{2} \|c - e - v\|^2 - \pi_T s_n$, a convex QP. The package solves it by

a primal–dual interior-point method with Mehrotra’s predictor–corrector (`admm.battery_qp`); the Newton system reduces to a tridiagonal one in the n balance duals, so an iteration costs $O(n)$. A warm start moves the previous solution 5% of each range off its bounds and floors its bound multipliers, which saves about a third of the iterations with the same answer.

C Numerical evidence

Every figure and table below is produced by the script named with it, on the package’s fixtures: three tariffs (flat, day/night, dynamic), $\Delta t = 0.25$ h. “Capture” is the fraction of the *available* savings (baseline minus optimum) a plan realises; the bill on a PV site crosses zero, so a ratio against it is meaningless.

C.1 How far from optimal

Figures 5–7 measure the two losses against exact references (`bench/make_theory_figs.py`). The DP battery’s loss is its grid; the coordinators’ loss is coordination plus the grids, measured against a MILP over the same model that closes to a 10^{-4} relative gap in 15–39 s (`bench/milp_full.py`; that time is a property of one untuned formulation, not of MILP solvers in general).

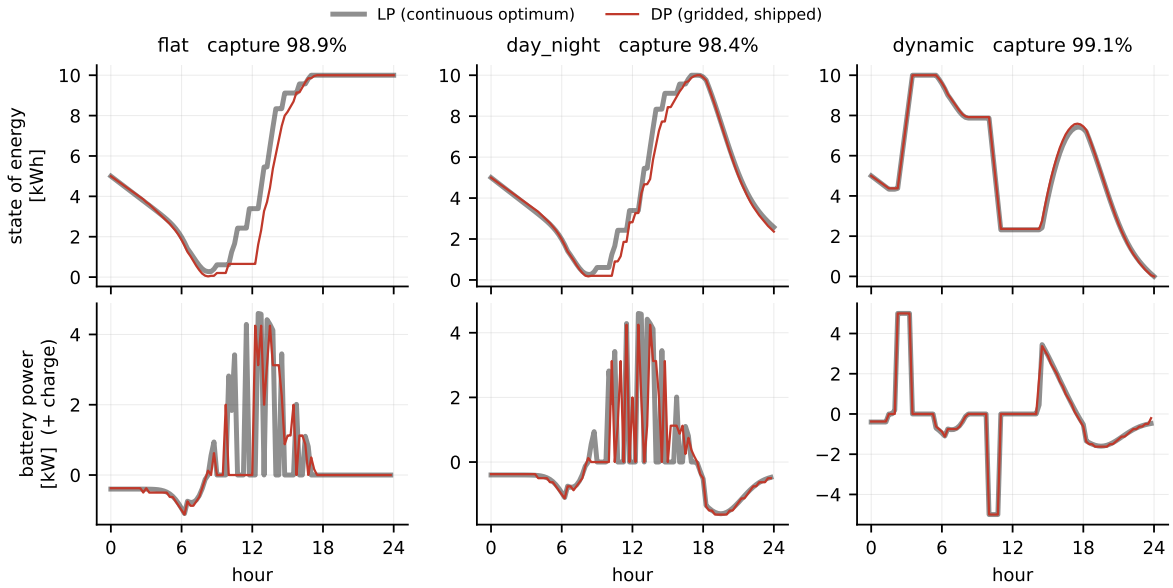


Figure 5: **The DP’s grid.** One battery planned by its DP (red) against its LP (grey), on the same model. The two agree on what to do and differ in the last increment of how much; capture 98.4–99.2%. A finer grid shrinks the difference, at a cost in solve time.

C.2 ADMM on the test sites

`bench/admm_stability.py` runs Algorithm 2 on 72 sites — the three tariffs, one to three batteries, a 7 kW import limit or none, 5 or 8 kW of PV, 10 or 20 kWh per battery, 48 h — and scores every plan against the Dantzig–Wolfe bound.

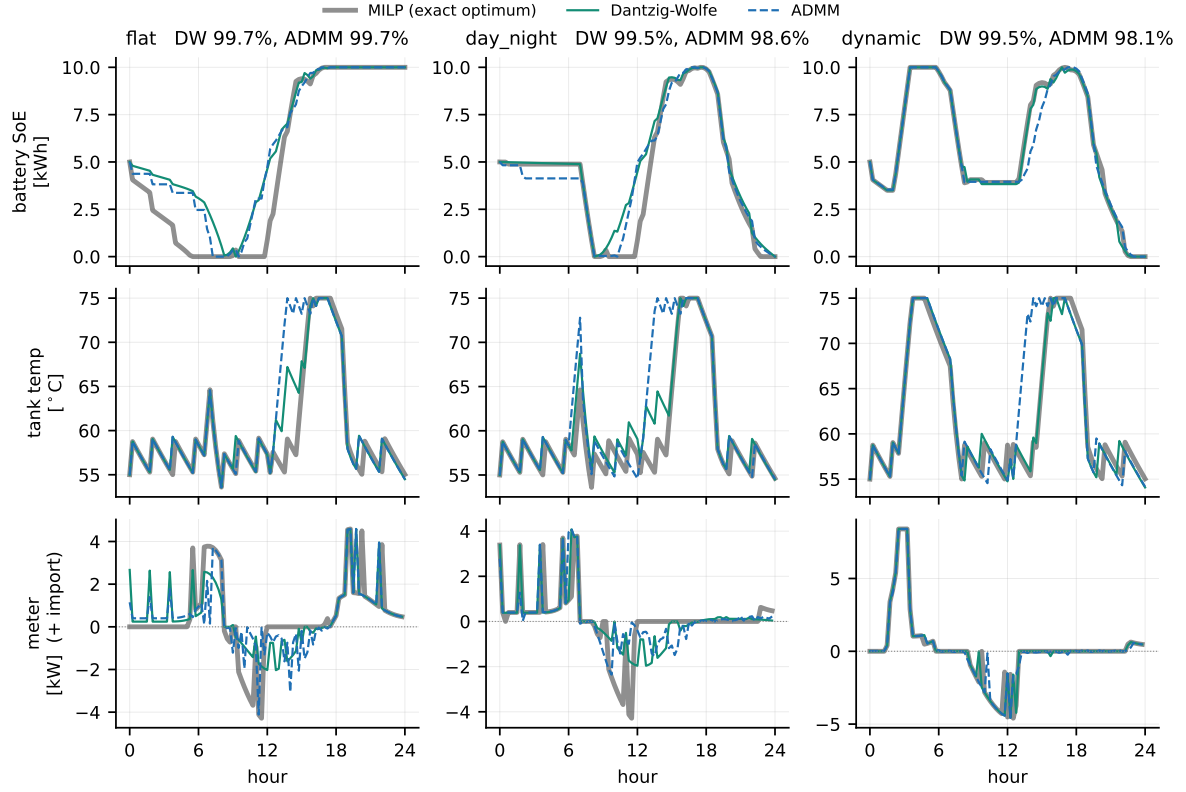


Figure 6: **Coordination.** Battery and on/off water heater, 24 h: Dantzig-Wolfe (green) and ADMM (blue, dashed) against a MILP over the same model (grey). The strategies have the same shape — discharge overnight, refill on PV, heat the tank into the surplus — and differ in timing and in how hot the tank is taken. Capture: Dantzig-Wolfe 99.5–99.7% in 0.6–0.7 s; ADMM 98.1–99.7% in 4–8 s.

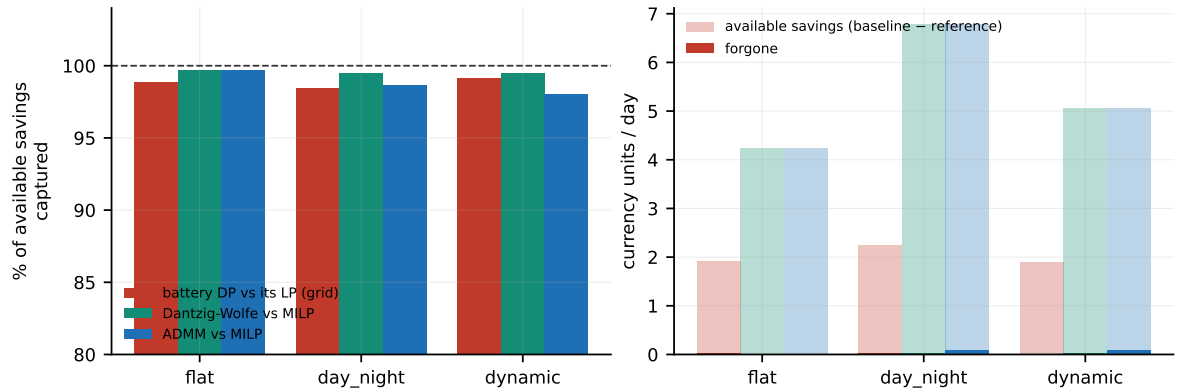


Figure 7: **What the percentages are percentages of.** Left: capture by tariff. Right: the same in money; the pale bar is the savings available over the baseline, the solid bar the part forgone. The first bar of each group is the battery-only site, the others the battery and water heater site.

	mean above bound	worst	converged	time per site
Dantzig–Wolfe	0.099	—	—	4.8 s
ADMM, DP steps (the default)	0.508	1.056	16 of 72	46 s
ADMM, LP battery steps	0.541	1.229	49 of 72	33 s
message passing, exact steps, then polish	0.451	—	72 of 72	13.3 s

The last row is proximal message passing as published: every device relaxed to a convex model and every step solved exactly as a QP (`bench/prior_art.py`, which needs `osqp`), followed by our polish; recovering runnable plans from its prices alone instead ends 3.9 above the bound. Converging does not make the on/off devices’ plans better: LP battery steps converge three times as often as DP steps, for plans slightly further from the bound. What closes the gap is choosing the on/off devices’ plans together, which Dantzig–Wolfe does. Seven of the 72 DP-step plans exceed the import limit, by at most 5 W, and pay for it.

What each departure is worth. On twelve of the sites (`bench/exchange_tune.py`), one change at a time against Algorithm 2 (the numbers are the departures of Section 4.4):

	above bound	worst	converged	time
as shipped	0.416	0.813	6 of 12	42 s
on/off devices not relaxed (2)	1.280	1.930	6	9 s
relaxed to quarters, not twelfths (2)	0.438	0.861	4	22 s
no polish (4)	1.987	3.983	6	41 s
no baseline fallback (4)	0.416	0.813	6	41 s
kink not rounded (5)	0.397	0.817	2	44 s
300 iterations, not 100 (5)	0.392	0.719	6	87 s
momentum with restart [9]	0.416	0.813	6	45 s
LP battery steps (1)	0.439	0.726	8	29 s

With exact steps (`bench/rho_sweep.py`), a starting ρ of 0.03–0.1 converged in a median of 80–115 iterations and one of 3 in 250–770; with a large ρ the residual test passed while the plan was still up to 0.78 from the optimum, which is one reason the best plan is kept rather than the last.

Many batteries (`bench/many_batteries.py`). Dynamic tariff, 7 kW import limit, 24 h, batteries of 10 to 28 kWh, DP steps. The imbalance over the last 20 iterations grows with the number of batteries and the power step of their grid:

batteries	power step	imbalance r_p , last 20 iterations	batteries moving at once
1	0.40 kW	0.06 (converged)	—
2	0.40 kW	0.36	1.5 of 2
4	0.40 kW	0.75	2.0 of 4
8	0.40 kW	2.46	8 of 8
8	0.20 kW	1.40	8 of 8
8	0.10 kW	0.76	8 of 8

With eight batteries the best plan, 1.0 above the bound, is found by iteration 13 and 300 iterations find nothing better. With LP battery steps the same site converges in 86 iterations (38 with the kink unrounded), to a plan 0.93 above the bound (0.84): the on/off devices still limit it. Dantzig–Wolfe comes within 0.05 of the bound in 2.3 s. With three batteries alone, LP steps and the kink unrounded, ADMM converges to the optimum (the master’s, which closes its gap on this site) to within its stopping tolerance:

tolerance ϵ^{abs}	iterations	above the optimum
10^{-3} (default)	110	0.049
10^{-4}	248	0.006
10^{-5}	847	0.001
10^{-6}	3000 (cap)	0.0001

Warm starts (`bench/warm_start.py`). Starting each LP battery step from its previous solution takes a third of the time out of those steps, with the same plans: three batteries alone solve in 0.5 s instead of 0.8 s. Starting a solve from the previous one’s best state after one setting changes (PV, battery size or import limit; twelve cases), the runnable plan came within 0.05 of the run’s best by iteration 21 on average against 65 cold, six runs converged instead of one, and the solves took 15 s instead of 21 s; the final plans were as good on average (0.222 above the bound against 0.223). One planning cycle later (forecast shifted by one slot; four cases), the plan was near its best by iteration 15 against 64, in a third less time, and ended 0.02 further from the bound on average.

Is it the on/off devices? Partly. Letting the water heater and HVAC run twelfths of a slot in the problem itself (`bench/relaxed_devices.py`) makes it nearly convex: Dantzig–Wolfe’s gap falls from 0.10 to zero within its grids. On those sites ADMM with DP steps (`bench/pmp_dp.py`) still converged on only 10 of 72 within 300 iterations, and ended 0.25 above the bound, against 0.51 on the real sites. The grid steps account for roughly half of ADMM’s distance from the bound; the on/off devices for the other half.

C.3 Scaling with the number of devices

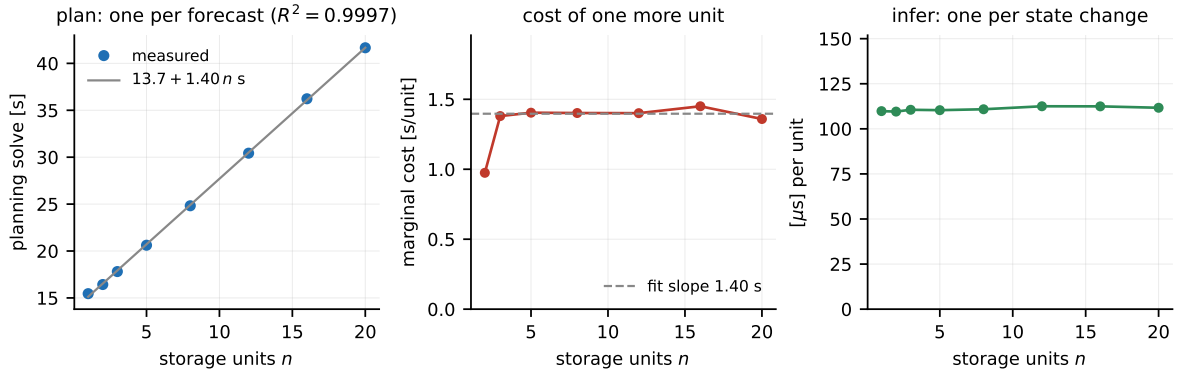


Figure 8: Solve cost against the number of batteries n , each with 100 energy and 41 power levels, over 96 slots, with the water heater and HVAC, planned by ADMM (100 iterations; `bench/run_scale.py`). **Left:** affine in n , $13.7 + 1.40n$ seconds ($R^2 = 0.9997$); the intercept is the thermal devices and fixed overhead. **Centre:** the cost of one more battery, flat at about 1.4 s. **Right:** the execution tier, 110–113 μ s per battery.

Twenty batteries plan in 42 s under ADMM and decide in 2.2 ms. Dantzig–Wolfe plans the same sites in 0.6 to 4.3 s from one battery to twenty: its iterations are few, and its master, which holds the batteries, grows with them.

C.4 The savings split against other rules

The same 13.677 as Table 4, split five ways (`bench/der_attribution.py`):

player	surplus at final π	proportional	A-S, solar a player	this rule	Shapley
solar	4.391	6.732	13.454	7.481	7.868
battery 1	0.940	1.442	-0.688	1.516	1.457
battery 2	1.134	1.739	-0.823	1.797	1.734
water heater	2.410	3.695	1.630	2.720	2.510
HVAC	0.045	0.069	0.104	0.163	0.109

Surplus at final π values each player's energy at the plan's meter prices, less its private-cost change; it accounts for only 8.9 of the saving. *Proportional* shares the whole saving in proportion to it. *Aumann-Shapley with solar as a player* prices slot by slot and misses storage moving PV energy in time, crediting almost everything to solar. On the three tariffs, with and without a 3 kW export limit, the rule came within 0.46 of the Shapley value in all six cases and within 0.13 for the batteries.

C.5 The testbed

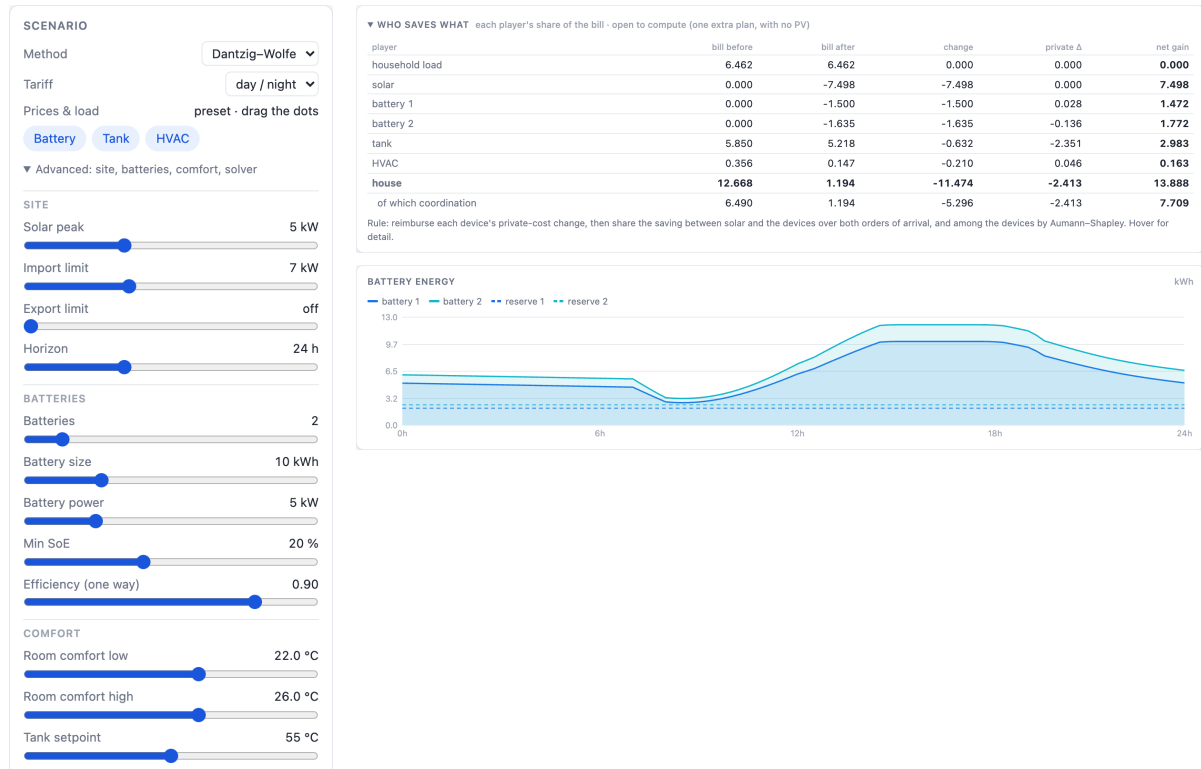


Figure 9: The testbed ([dw/gui/](#), also a standalone page that runs in the browser) plans a site with either coordinator and runs the code the tests run. Day/night tariff, two batteries with a 20% reserve, Dantzig-Wolfe. Left: the controls. Right: the ledger of Section 7 and the batteries' stored energy, reserves dashed. For ADMM the page offers each battery's step (DP or LP) and a cold or warm start, and a solve can be paused, showing the best plan so far, and resumed.

Figure 9 shows the testbed. The ledger there differs from Table 4, which uses an on/off water heater; on the page it modulates and sits in the master.

D Curtailment and the price devices see

Proposition 1. *Let curtailment be free up to the PV output g_t , and let no plan export more than g_t in a slot where $c_t^{\text{exp}} < 0$. Then planning each device's best response priced at $\max(c_t^{\text{exp}}, 0)$*

and curtailing afterwards is exact: it is the best response to M_t . Priced at the raw c_t^{exp} it is not.

Proof. Curtailment raises z_t only where the house exports, and reduces export one for one, so it changes neither the residual load nor any device's feasible set. It changes the value of the alternatives. With curtailment available, a kWh of PV surplus is never exported at a negative price, because discarding it costs nothing; so the revenue a device forgoes by absorbing a unit of surplus is $\max(c_t^{\text{exp}}, 0)$, which is the slope of M_t there. Priced at a negative c_t^{exp} , a device is credited $|c_t^{\text{exp}}|$ per kWh for absorbing surplus that could be discarded for free, and pays real conversion losses for a benefit that does not exist. Where $c_t^{\text{exp}} \geq 0$ the two prices agree. $\square$

Measured on the day/night fixture with a -0.10 midday export price, a battery priced at the raw tariff costs 0.31 more per day, and cycles 7.3 kWh more, than priced at $\max(c^{\text{exp}}, 0)$; the difference grows linearly as the price falls and vanishes for $c^{\text{exp}} \geq 0$. An export *cap* needs no correction: its breach price already prices it, and absorbing surplus into storage beats discarding it whenever there is headroom.